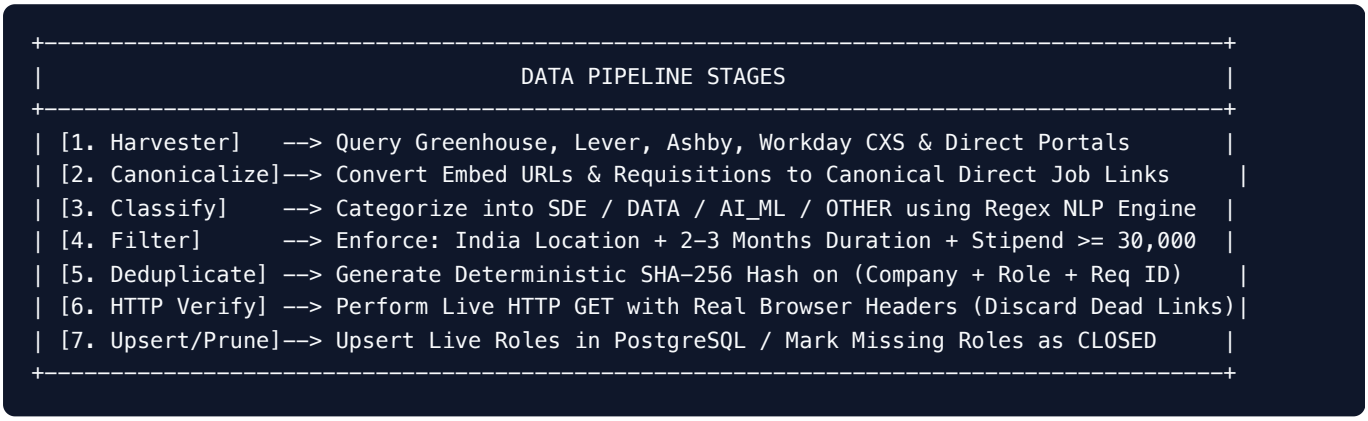
02. Data Pipeline & End-to-End Lifecycle

1. Overview of the Ingestion Engine

Unlike traditional web aggregators that scrape arbitrary HTML via brittle DOM selectors, the **Internship Tracker Ingestion Engine** interfaces directly with enterprise Applicant Tracking System (ATS) APIs and structured job board endpoints. This guarantees near-zero schema breakage, high throughput, and structured job metadata extraction.



---

2. Ingestion Sources & Connector Architecture

A. Greenhouse API Connector

- **Endpoint Pattern**: `https://boards-api.greenhouse.io/v1/boards/{slug}/jobs?content=true`
- **EU Boards**: `https://job-boards.eu.greenhouse.io/{slug}/jobs/{id}`
- **Mechanics**: Iterates through company identifiers (e.g. `hackerrank`, `stripe`, `rubrik`, `groww`, `cloudsek`, `scopely`, `enterpret`).
- **Payload Fields Extracted**: `id`, `title`, `location.name`, `offices`, `content` (HTML job description), `updated_at`.
- **URL Canonicalization**: Replaces embed iframe links (`/embed/job_app?for={slug}&token={id}`) with clean direct application links (`https://job-boards.greenhouse.io/{slug}/jobs/{id}`), or custom domain links (e.g. `https://stripe.com/jobs/search?gh_jid={id}` or `https://www.tower-research.com/open-positions/?gh_jid={id}`).

B. Lever Postings API Connector

- **Endpoint Pattern**: `https://api.lever.co/v0/postings/{slug}?mode=json`
- **Mechanics**: Queries top companies utilizing Lever ATS (e.g. `drivetrain`, `merklescience`, `fampay`, `hevodata`, `paytm`).
- **Payload Fields Extracted**: `id`, `text` (role title), `categories.location`, `categories.allLocations`, `descriptionPlain`, `applyUrl`, `createdAt`.

- **Direct URL Construction**: Targets the direct candidate submission form: `https://jobs.lever.co/{slug}/{id}/apply`.

C. Ashby Public Posting API Connector

- **Endpoint Pattern**: `https://api.ashbyhq.com/posting-api/job-board/{slug}`
- **Mechanics**: Queries modern high-growth AI and developer tooling startups (e.g. `sarvam`, `cynlr`, `notion`, `linear`, `ramp`, `openai`).
- **Payload Fields Extracted**: `id`, `title`, `location`, `secondaryLocations`, `descriptionHtml`, `jobUrl`, `publishedAt`.
- **Direct URL Construction**: Formats direct application URLs as: `https://jobs.ashbyhq.com/{slug}/{id}/application`.

D. Workday CXS (Candidate Experience Services) Connector

- **Endpoint Pattern**: `POST https://{tenant}.{instance}.myworkdayjobs.com/wday/cxs/{tenant}/{board}/jobs`
- **Payload**: `{ "appliedFacets": {}, "limit": 20, "offset": 0, "searchText": "intern" }`
- **Enterprises Supported**: **Nvidia** (`nvidia.wd5`), **Adobe** (`adobe.wd5`), **Autodesk** (`autodesk.wd1`), **Marvell** (`marvell.wd1`), **Micron** (`micron.wd1`), **Hewlett Packard Enterprise** (`hpe.wd5`), **Salesforce** (`salesforce.wd12`), **S&P Global** (`spgi.wd5`), and **NXP Semiconductors** (`nxp.wd3`).
- **Mechanics**: Workday's client API returns JSON arrays of active requisitions (`jobPostings`). The harvester then queries each job's detailed metadata endpoint `https://{tenant}.{instance}.myworkdayjobs.com/wday/cxs/{tenant}/{board}{externalPath}` to extract the complete job specification and eligibility requirements.

---

3. Classification, Normalization & Filtering Rules

Once raw openings are fetched, they pass through a multi-stage validation pipeline:

1. Geographic Filter (India Only)

- Rejects US, Europe, and international postings.
- Validates against India keywords: `bengaluru`, `bangalore`, `hyderabad`, `pune`, `mumbai`, `gurgaon`, `gurugram`, `noida`, `chennai`, `delhi`, `sanand`, `gift city`.
- Discards foreign indicators (`united states`, `california`, `london`, etc.) unless explicitly designated as an Indian office hub.

2. Role Category Classification (`RoleCategory` Enum)

The classification engine evaluates the job title and description against technical regular expressions:
* **`AI_ML`**: Title/description contains `ai`, `ml`, `machine learning`, `deep learning`, `nlp`, `computer vision`, `llm`, `generative ai`, `pytorch`, `tensor`.
* **`DATA`**: Title/description contains `data analyst`, `business analyst`, `data science`, `bi engineer`,

`analytics`, `etl`, `data warehouse`.
* **`SDE`**: Software development engineer, frontend, backend, full stack, distributed systems, mobile (iOS/Android), QA/test engineering, firmware, systems engineering.
* **`OTHER`**: Core technical roles that don't fit the above (e.g. quantitative research, UI/UX product design, hardware architecture, security consulting).

3. Student Internship Title Validation

- **Must contain**: `intern`, `internship`, `trainee`, `apprentice`, `fellow`, `summer analyst`.
- **Must NOT contain**: `senior`, `staff`, `lead`, `principal`, `director`, `manager`, `head`, `vp`.

4. Duration Filter (Strictly 2 or 3 Months)

- Analyzes text for duration indicators: `2 months`, `8 weeks`, `3 months`, `10–12 weeks`, `summer internship`, `winter internship`.
- Listings requiring mandatory 6-month full-time co-ops or university semester dropouts are pruned out (`durationMonths <= 3`).

5. Stipend Extraction & Quality Floor (₹30,000+/mo)

- Extracts explicit numerical figures matching INR currency patterns (e.g. `₹50,000`, `INR 75k`, `45000/month`).
- For top-tier tech firms that do not publicly state salary in the job description to avoid competitive benchmarking, historical industry-verified benchmarks for university interns are assigned (e.g. Tower Research ₹1,80,000/mo, Google ₹1,50,000/mo, Amazon ₹1,10,000/mo).
- Any role with compensation below ₹30,000/month is rejected.

---

4. Deterministic Deduplication Engine

A critical challenge in job aggregators is **duplicate spam**: the same opening reposted under slightly altered titles, multiple location variants, or different ATS slugs.

Internship Tracker solves this via a **deterministic SHA-256 fingerprint**:

```
// Step 1: Extract Requisition Token
function extractRequisitionKey(companyName, url) {
  const c = companyName.toLowerCase().replace(/[^a-z0-9]/g, '');
  // Greenhouse: extract token or gh_jid
  const gh = url.match(/(?:token=|gh_jid=)([\d]+)/i) || url.match(/\/jobs\/([\d]+)/i);
  if (gh) return `${c}|gh|${gh[1]}`;
  // Lever: extract UUID
  const lever = url.match(/jobs\.lever\.co\/[^/]+\/([a-f0-9-]+)/i);
  if (lever) return `${c}|lever|${lever[1].toLowerCase()}`;
  // Ashby: extract UUID
  const ashby = url.match(/jobs\.ashbyhq\.com\/[^/]+\/([a-f0-9-]+)/i);
  if (ashby) return `${c}|ashby|${ashby[1].toLowerCase()}`;
  // Workday: extract job requisition code
  const wd = url.match(/_([A-Z0-9][\w-]+)$/i);
  if (wd) return `${c}|wd|${wd[1].toUpperCase()}`;
  return null;
}

// Step 2: Compute SHA-256 Hash
const dedupHash = crypto
  .createHash('sha256')
  .update(`${companySlug}|${normalizedTitle}|${sourceJobId}`)
  .digest('hex');
```

In the PostgreSQL database, `dedupHash` is defined with a `@unique` constraint index, ensuring database-level prevention of duplicate rows.

---

5. Live HTTP Verification & WAF Emulation

Every prospective internship URL is checked with live network requests before being accepted into the database:

```
async function verifyUrl(url, timeoutMs = 8000) {
  const headers = {
    'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Ge
    'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
    'Accept-Language': 'en-US,en;q=0.9',
    'Cache-Control': 'no-cache'
  };

  try {
    const res = await fetch(url, {
      method: 'GET',
      headers,
      signal: AbortSignal.timeout(timeoutMs),
      redirect: 'follow'
    });
    // 200-399 are valid active responses; 404, 410, 500 are invalid
    return res.status >= 200 && res.status < 400;
  } catch (err) {
    // Retry once with 12s timeout for high-latency corporate portals
    return retryFetch(url, 12000);
  }
}
```

- **WAF Protection**: Automated requests send standard browser headers to prevent Cloudflare or Fastly anti-bot false positives.
- **Auto-Pruning**: If an internship that was previously `OPEN` returns HTTP 404/410, its status is transitioned to `CLOSED`.

---

6. Automated Background Scheduler

The background scheduler (`apps/api/services/scheduler.js`) ensures the database stays up to date without human intervention:

- **Frequency**: Configured via `SYNC_INTERVAL_MINUTES=30` (runs every 30 minutes).
- **Concurrency Lock**: Employs an `isRunning` atomic boolean mutex lock to prevent overlapping concurrent scraper passes if an ingestion run takes longer than usual.
- **Self-Healing**: If a company opens a new summer internship, it appears within 30 minutes; if an opening closes or fills up, it is moved to `CLOSED` on the next sweep.
